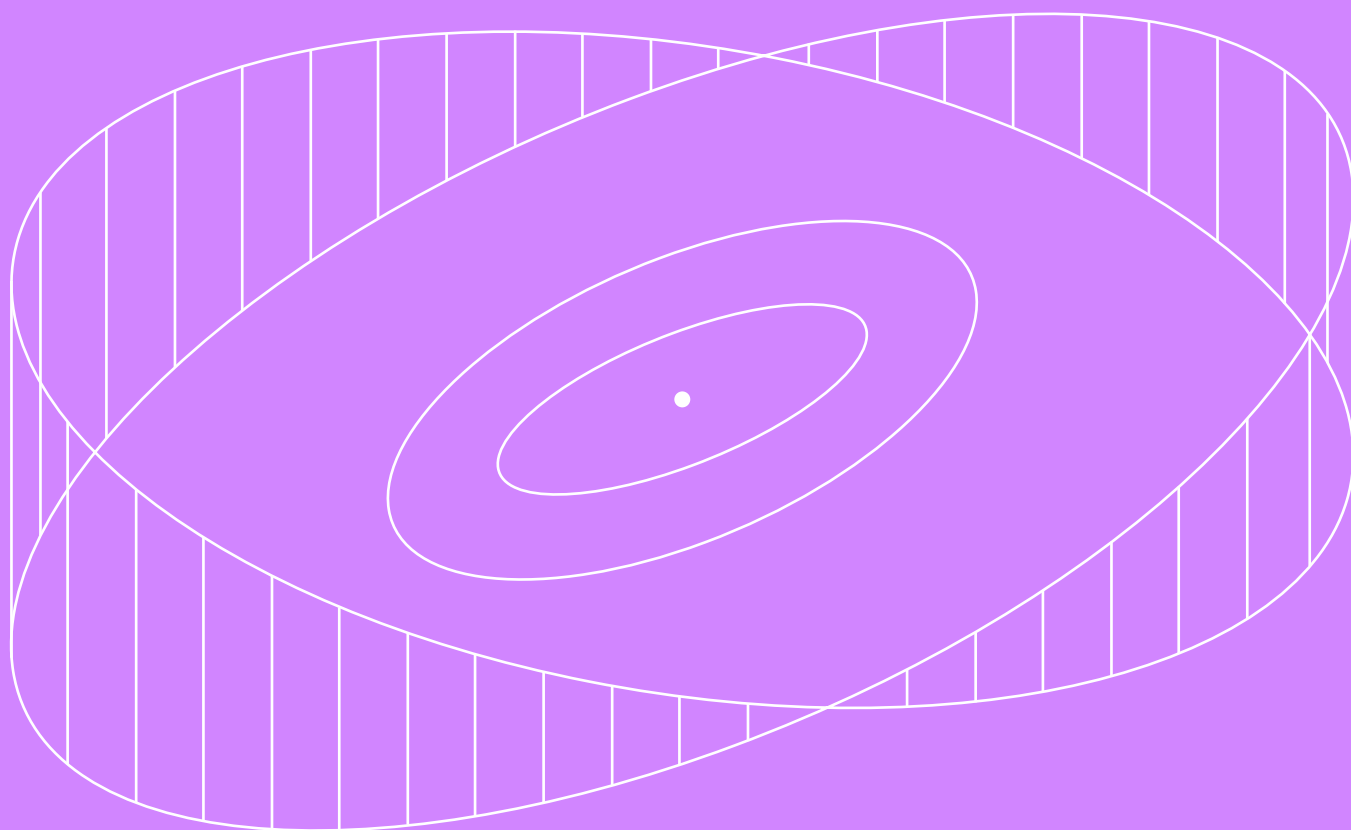


Основы DDL. Модификация таблиц. Представления



Введение

Сегодня поговорим про процесс управления базами данных, про категории операторов SQL, которые формируют структуру взаимодействия с данными, их изменением и управлением разрешениями. В современном мире данных, где объёмы информации увеличиваются с каждым днём, управление данными становится важной задачей для организаций любого размера. Тут ключевую роль играет DDL (Data Definition Language) — инструментарий для управления структурой данных в реляционных базах данных.

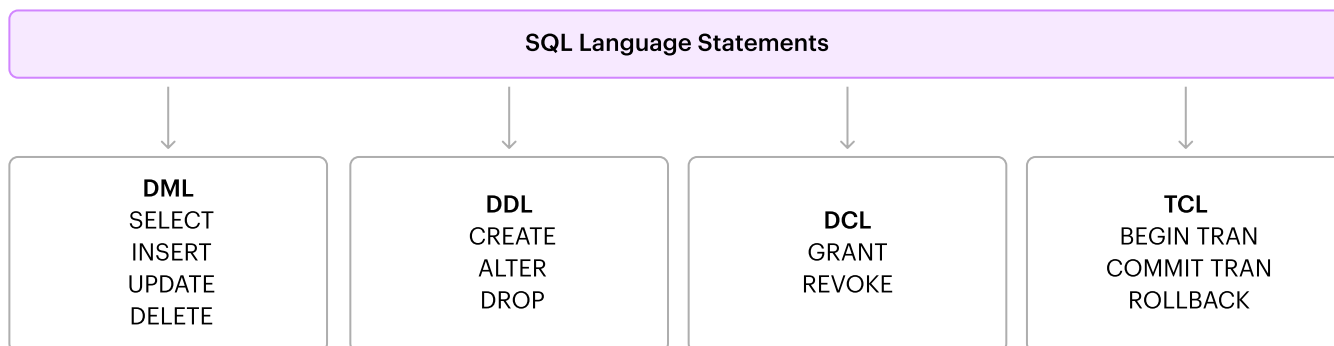
Одной из основных задач инженера данных является создание адекватной модели данных, которая отражает бизнес-логику и обеспечивает целостность информации. Неправильное определение структуры данных может привести к неэффективной работе с базой, что, в свою очередь, может негативно сказаться на производительности приложений и точности анализа данных. Именно поэтому необходимо быть знакомым с инструментарием, который помогает избежать всех этих потенциальных проблем.

Важность DDL также заключается в его роли в автоматизации процессов управления данными, таких как создание и обновление схем баз данных. Современные инструменты для работы с базами данных предоставляют разработчикам возможность использовать DDL для управления структурой данных через программы, например DBeaver, SQL Server Management Studio или pgAdmin, что снижает вероятность ошибок, связанных с ручным управлением.

В данном уроке мы рассмотрим основные команды DDL, их применение на практике, а также лучшие практики и подходы к проектированию структур данных.

Определение DDL и основные команды

Язык структурированных запросов (SQL) включает несколько подмножеств команд, каждая из которых предназначена для выполнения определённых задач в управлении базами данных. Подробнее все разделы представлены на картинке ниже.



Ранее мы познакомились с командами по работе с базами данных (на картинке выше — блок DML), однако для эффективного функционирования базы данных также необходимо уметь создавать объекты баз данных, добавлять и изменять данные, представленные в них, а также корректно определять структуру самой базы. Здесь особую роль играет язык определения данных, известный как DDL.

DDL является одним из основных компонентов SQL, предоставляя возможность создавать, изменять и удалять объекты внутри базы данных. В этом разделе мы рассмотрим ключевые команды DDL и их значимость для администрирования баз данных.

КОМАНДА CREATE

Оператор **CREATE TABLE** в SQL является одной из основных команд языка определения данных (DDL), используемой для создания новых таблиц в базе данных. С помощью этого оператора ты можешь определить структуру таблицы, включая названия и типы столбцов, а также задать ограничения на данные, такие как первичные ключи, уникальные индексы и ограничения на значения.

Синтаксис операции **CREATE TABLE** включает указание имени таблицы (с указанием схемы, в которой создаётся таблица) и описания её структуры в виде списка столбцов. Каждый столбец должен иметь определённое имя и тип данных, например **VARCHAR**, **INT**, **DATE** и так далее.

Синтаксис:

```
CREATE TABLE {название таблицы} (  
    столбец1 тип данных столбца1 параметр для столбца1  
    , столбец2 тип данных столбца2 параметр для столбца2  
    , ...  
    , столбецN тип данных столбцаN параметр для столбцаN  
);
```

Кроме того, ты можешь указать различные параметры, такие как **NOT NULL**, которые гарантируют, что в столбце не может находиться значение **NULL**, или **UNIQUE**, что обозначает уникальные ограничения, которые предотвращают дублирование значений в заданных столбцах.

Также существуют параметры **DEFAULT** и **CHECK**, которые помогают обеспечивать целостность данных и управлять значениями, которые могут быть вставлены в столбцы.

→ Параметр **DEFAULT** задаёт значение по умолчанию для столбца в таблице, если при вставке данных не указано значение для данного столбца. Это позволяет избежать наличия **NULL**-значений и гарантирует, что в столбце будет находиться заранее определённое значение, если пользователь не предоставит его.

```
CREATE TABLE имя_таблицы (  
    имя_столбца тип_данных DEFAULT значение  
    , ...  
);
```

→ Параметр **CHECK** используется для определения ограничения на значения, которые могут быть вставлены в определённый столбец. Это позволяет установить правила, которые данные должны соблюдать, и тем самым обеспечивает их корректность и целостность.

```
CREATE TABLE имя_таблицы (  
    имя_столбца тип_данных CHECK (условие)  
    , ...  
);
```

Также стоит отметить такие параметры, как **PRIMARY KEY** и **FOREIGN KEY**. Данные параметры играют важную роль в обеспечении структуры, целостности и взаимосвязей между таблицами. Давай разберём каждую из этих концепций подробнее.

PRIMARY KEY

Это ограничение, которое используется для уникальной идентификации каждой записи в таблице. Каждая таблица может иметь только один первичный ключ, состоящий из одного или нескольких столбцов. Основные характеристики первичного ключа:

1. **Уникальность.** Значение первичного ключа должно быть уникальным для каждой записи в таблице, что позволяет избежать дублирования.
2. **NOT NULL.** Столбцы, определённые как часть первичного ключа, не могут содержать значение NULL. Это гарантирует, что каждая запись имеет чёткий и однозначный идентификатор.
3. **Индексация.** В большинстве систем управления базами данных (СУБД) создание первичного ключа автоматически создаёт индекс, что ускоряет операции поиска.

FOREIGN KEY

Это ограничение, которое используется для создания связи между двумя таблицами. Оно обеспечивает ссылочную целостность данных, гарантируя, что значение в одном столбце (или группе столбцов) соответствует значению в первичном ключе другой таблицы. Основные характеристики внешнего ключа:

1. **Ссылочная целостность.** Внешний ключ позволяет контролировать, что каждая запись ссылается на существующую запись в родительской таблице. Если ты попытаешься вставить значение внешнего ключа, которое отсутствует в родительской таблице, база данных не позволит это сделать.
2. **Обновление и удаление.** При изменении или удалении записи в родительской таблице можно настроить действие для связанных записей в дочерней таблице, например, с помощью каскадного обновления (**ON UPDATE CASCADE**) или каскадного удаления (**ON DELETE CASCADE**).

ПРИМЕР

Создать таблицу `orders`, в которой должны быть следующие столбцы и ограничения:

- `order_id` — первичный ключ, тип данных **VARCHAR**;
- `order_sum` — не может принимать значение `NULL`, тип данных **FLOAT**;
- `product_id` — используется для связи с другой таблицей, тип данных **VARCHAR**.

В терминах языка SQL запрос будет выглядеть так:

```
CREATE TABLE orders (  
    order_id VARCHAR PRIMARY KEY  
    , order_sum FLOAT NOT NULL  
    , product_id VARCHAR FOREIGN KEY  
);
```

КОМАНДА DROP

Команда **DROP TABLE** в SQL используется для удаления из базы данных таблицы, а также всех данных, хранящихся в ней. Эта команда выполняется с большой осторожностью, так как все записи и структура таблицы будут безвозвратно утрачены. Рассмотрим более подробно синтаксис и особенности использования команды **DROP TABLE**.

Общий синтаксис команды **DROP TABLE** выглядит следующим образом:

```
DROP TABLE имя_таблицы;
```

Особенности

1. **Безвозвратное удаление.** После выполнения команды **DROP TABLE** восстановить данные невозможно, если нет резервной копии. Поэтому всегда рекомендуется предварительно проверять, действительно ли нужно удалить таблицу.
2. **Удаление нескольких таблиц.** Если ты хочешь удалить несколько таблиц одновременно, ты можешь указать их имена через запятую:

```
DROP TABLE таблица1, таблица2, таблица3;
```

3. **Проверка существования таблицы.** Некоторые СУБД поддерживают возможность проверки существования таблицы перед её удалением. Это можно реализовать с помощью конструкции **IF EXISTS**:

```
DROP TABLE IF EXISTS имя_таблицы;
```

Эта команда удалит таблицу только в том случае, если она существует, что помогает избежать ошибок.

4. Когда ты пытаешься удалить таблицу, на которую ссылается другая таблица через внешний ключ, СУБД обычно выдаёт ошибку. Чтобы избежать этой ошибки, ты можешь использовать параметр **CASCADE** при удалении таблицы. Это сообщает СУБД автоматически удалить все записи в связанных таблицах, что позволяет безопасно удалить таблицу.

```
DROP TABLE customers CASCADE;
```

КОМАНДА ALTER

Команда **ALTER** в SQL используется для изменения структуры существующих объектов базы данных, таких как таблицы, представления (о них поговорим чуть позже) и базы данных. Она предоставляет возможность добавлять, изменять или удалять столбцы, а также управлять индексами и ограничениями. Важно отметить, что для использования команды **ALTER** необходимо быть или владельцем таблицы, или иметь соответствующие доступы.

Рассмотрим подробнее синтаксис команды **ALTER**, её функции и примеры использования.

Команда **ALTER TABLE**, которая применяется для изменения таблиц, имеет несколько основных форм:

1. **Добавление нового столбца:**

```
ALTER TABLE имя_таблицы ADD имя_столбца тип_данных  
[ограничения] ;
```

2. **Изменение существующего столбца:**

```
ALTER TABLE имя_таблицы MODIFY имя_столбца новый_тип_данных  
[новые_ограничения] ;
```

3. Удаление столбца:

```
ALTER TABLE имя_таблицы DROP имя_столбца ;
```

4. Добавление ограничения:

```
ALTER TABLE имя_таблицы ADD CONSTRAINT имя_ограничения  
тип_ограничения (имя_столбца) ;
```

5. Удаление ограничения:

```
ALTER TABLE имя_таблицы DROP CONSTRAINT имя_ограничения ;
```

КОМАНДА TRUNCATE

Команда **TRUNCATE TABLE** в SQL используется для быстрого удаления всех строк из таблицы и сохраняет при этом структуру таблицы.

```
TRUNCATE TABLE имя_таблицы ;
```

Особенности

Некоторые особенности будут проведены в сравнении с командой **DELETE**, которую подробнее рассмотрим в следующем разделе.

1. **Быстродействие.** **TRUNCATE TABLE** обычно выполняется быстрее, чем **DELETE**, потому что не активирует триггеры и не записывает удаляемые строки в журнал операций (в некоторых СУБД). Это делает **TRUNCATE** более подходящей для ситуаций, когда нужно быстро очистить таблицу.
2. **Не поддерживает условия.** В отличие от **DELETE**, команда **TRUNCATE** не позволяет указывать условия и удаляет все строки без возможности фильтрации.
3. **Сохранение структуры таблицы.** Хотя все данные будут удалены, структура таблицы (столбцы, индексы, ограничения) останется на месте. Это позволяет сразу же вставлять новые данные в таблицу после её очистки.
4. **Особенности использования.** **TRUNCATE** нельзя использовать с таблицей, на которую по внешнему ключу ссылаются другие таблицы, если только и эти таблицы не опустошаются этой же командой.

Основные команды добавления, изменения и удаления строк в таблице

Также существует ряд операторов, которые позволяют производить преобразования не с целой таблицей, а работать со строками.

Примеры будем разбирать на следующей таблице **orders**.

order_id	customer_id	merchant_id	order_amount	order_date	delivery_date	order_status	delivery_status
1	1	101	1000	2023-01-01	2023-01-03	paid	delivered
2	2	102	2000	2023-01-02	2023-01-04	paid	delivered
3	1	103	3000	2023-01-03	2023-01-06	completed	in transit
4	3	104	1500	2023-01-04	2023-01-05	completed	delivered
5	2	101	5000	2023-01-05	2023-01-07	paid	delivered

INSERT

Помимо извлечения данных из БД, существует ряд других операций. Например, мы можем записывать новые данные в нашу БД. Сформируем запрос для данного кейса:

```
ВСТАВЬ В ТАБЛИЦУ 'заказы' ЗНАЧЕНИЯ 'id заказа' = 6,  
'сумма заказа' = 500, 'дата заказа' = '2023-06-01'
```

В терминах языка SQL запрос будет выглядеть так:

```
INSERT INTO orders (order_id, order_amount, order_date)  
VALUES  
(6, 500, '2023-06-01')
```

Также важно отметить, что производить запись данных можно не по одной строке, как в примере выше, а передавать данные из другой таблицы при помощи операторов **SELECT** и **FROM**, которые мы разбирали ранее.

ПРИМЕР

Хотим добавить в таблицу `orders` информацию, которая хранится в таблице `new_orders` (считаем, что колонки в этих таблицах полностью идентичны).

В терминах языка SQL запрос будет выглядеть так:

```
INSERT INTO orders
SELECT *
FROM new_orders
```

Таким образом, можем записать общий вид синтаксиса оператора **INSERT**:

```
INSERT INTO имя_таблицы [(поле_таблицы, ...)]
VALUES (значение_поля_таблицы, ...)
| SELECT поле_таблицы, ... FROM имя_таблицы ...
```

DELETE

С помощью SQL можно не только добавлять или читать данные, но и удалять их, используя оператор **DELETE**.

УДАЛИТЬ всё ИЗ ТАБЛИЦЫ "заказы" ГДЕ "статус заказа" = "ошибка"

В терминах языка SQL запрос будет выглядеть так:

```
DELETE
FROM orders
WHERE order_status = 'error'
```

UPDATE

Также существует возможность редактировать текущие записи при помощи оператора **UPDATE**.

ЗАМЕНИТЬ "статус заказа" на "отменен" В ТАБЛИЦЕ "заказы" ГДЕ "ID заказа" = 5

В терминах языка SQL запрос будет выглядеть так:

```
UPDATE orders
SET order_status = 'canceled'
WHERE order_id = 5
```

Например, если после выполнения запроса с оператором **UPDATE** обратимся к таблице `orders` и выведем из неё все данные, то получим следующее:

order_id	customer_id	merchant_id	order_amount	order_date	delivery_date	order_status	delivery_status
1	1	101	1000	2023-01-01	2023-01-03	paid	delivered
2	2	102	2000	2023-01-02	2023-01-04	paid	delivered
3	1	103	3000	2023-01-03	2023-01-06	completed	in transit
4	3	104	1500	2023-01-04	2023-01-05	completed	delivered
5	2	101	5000	2023-01-05	2023-01-07	canceled	delivered

Представление данных (view)

Начнём с того, что это такое. Представление — это объект базы данных, который создаётся на основе выполнения запроса, заданного оператором **SELECT**, и отображается в момент обращения к представлению.

Представления иногда называют виртуальными таблицами. Это связано с тем, что они доступны пользователям как таблицы, но не содержат данных сами по себе; вместо этого они извлекают данные из связанных таблиц в момент запроса. Если данные в базовых таблицах изменяются, пользователи получают актуальные результаты при обращении к соответствующему представлению.

ПРЕИМУЩЕСТВА ИСПОЛЬЗОВАНИЯ ПРЕДСТАВЛЕНИЙ

- 1. Гибкость в управлении правами доступа.** Представления позволяют настраивать права доступа к данным, предоставляя доступ не к таблицам, а к представлениям. Это особенно удобно в случаях, когда необходимо предоставить пользователю доступ к отдельным строкам и столбцам таблицы или результатам определённых операций с данными.
- 2. Разделение логики хранения данных и программного обеспечения.** Использование представлений позволяет изменять структуру данных без необходимости менять программный код. Достаточно создать представления, которые выступают аналогами ранее использованных таблиц. Это полезно, когда модификация кода невозможна или когда к одной базе данных обращается несколько приложений с различными требованиями по структуре данных.
- 3. Удобство в использовании.** Представления автоматизируют доступ к определённым строкам и/или столбцам, а также позволяют извлекать данные из нескольких таблиц и преобразовывать их с помощью различных функций, упрощая взаимодействие с данными.

КОМАНДЫ ПО РАБОТЕ С ПРЕДСТАВЛЕНИЯМИ

С представлениями можно производить следующие действия:

→ Создавать представления:

```
CREATE VIEW имя_представления AS  
SELECT выражение_таблиц  
WHERE условие ;
```

→ Создавать представления:

```
CREATE OR REPLACE VIEW имя_представления AS  
SELECT новое_выражение_таблиц  
WHERE новое_условие ;
```

→ Удалять представления:

```
DROP VIEW имя_представления ;
```

Заключение

В этом уроке ты познакомился с основными командами DDL. Теперь ты сможешь не только считывать данные из таблиц, но и создавать собственные таблицы, изменять их, а при необходимости и удалять.